Razor templates somewhat easier to read that than the pure plain text.

Although the ability to use loops and conditionals is powerful—they're one of the advantages of Razor over static HTML—they also add to the complexity of your view. Try to limit the amount of logic in your views to make them as easy to understand and maintain as possible.

A common trope of the ASP.NET Core team is that they try to ensure you "fall into the pit of success" when building an application. This refers to the idea that by default, the *easiest* way to do something should be the *correct* way of doing it. This is a great philosophy, as it means you shouldn't get burned by, for example, security problems if you follow the standard approaches. Occasionally, however, you may need to step beyond the safety rails; a common use case is when you need to render some HTML contained in a C# object to the output, as you'll see in the next section.

17.3.3 Rendering HTML with Raw

In the previous example, we rendered the list of tasks to HTML by writing the `string task` using the `@task` Razor expression. But what if the `task` variable contains HTML you want to display, so instead of `"Check oil"` it contains `"<strong>Check oil</strong>"`? If you use a Razor expression to output this as you did previously, you might hope to get this:

```
<li><strong>Check oil</strong></li>
```

But that's not the case. The HTML generated comes out like this:

```
<li>&lt;strong&gt;Check oil&lt;/strong&gt;</li>
```

Hmm, looks odd, right? What's happened here? Why did the template not write your variable to the HTML, like it has in previous examples? If you look at